นี่คือโค้ด PPO V2

```
#
================================================================================
====
# PATCH V9.2 — Fully Audited Open-to-Open DRL Multi-Asset Portfolio
Allocation
# Features:
#   - Open-to-Open Execution Timeline (Hold Open(t+1) to Open(t+2))
#   - Zero Observation Data-Leakage (Weights in obs drifted to Close(t+1)
only)
#   - No-Trade Band to minimize turnover churning
#   - Strictly Aligned Benchmark Suite (Friction & Timeline matched)
#   - 8-Point Deterministic Unit & Financial Audit Suite
#
================================================================================
====

import os
import random
import warnings
from dataclasses import dataclass
from pathlib import Path

import gymnasium as gym
import numpy as np
import pandas as pd
import pandas_datareader.data as web
import quantstats as qs
import ta
import torch
import torch.nn as nn
import yfinance as yf
from gymnasium import spaces
from stable_baselines3 import PPO
from stable_baselines3.common.callbacks import EvalCallback
from stable_baselines3.common.vec_env import DummyVecEnv, SubprocVecEnv,
VecNormalize

warnings.filterwarnings("ignore")
```

```
#
================================================================================
====
# 0. CONFIG
#
================================================================================
====

@dataclass(frozen=True)
class Config:
    tickers = ("DIA", "QQQ", "SPY")

    initial_amount = 1_000_000.0
    transaction_cost_pct = 0.0015  # 15 bps ต่อมูลค่าซื้อขาย

    seed = 42
    device = "cpu"

    max_cpu = 8
    reserve_cpu = 1

    target_rollout = 8192
    batch_size = 256

    total_timesteps = 1_000_000
    eval_every = 50_000
    n_eval_episodes = 1

    global_fetch_start = "1996-01-01"
    global_end = "2026-07-29"

    # FRED Availability Assumptions
    fed_funds_lag_days = 35
    m2_lag_days = 60

    # Reward Shaping
    dd_penalty_mult = 2.0
    cash_drag_threshold = 0.30
```

```
    cash_drag_strength = 0.05
    high_vix_threshold = np.log(25.0)  # Absolute Log-Level ของ VIX (> 25)
    high_vix_drag_multiplier = 0.25

    # PPO Hyperparameters
    learning_rate = 0.0002
    n_epochs = 10
    gamma = 0.99
    gae_lambda = 0.90
    vf_coef = 1.0
    ent_coef = 0.01
    clip_range = 0.2
    max_grad_norm = 0.5
    target_kl = 0.02
    log_std_init = -1.0

    # No-trade band (Threshold turnover)
    min_turnover_threshold = 0.01


CFG = Config()

WALK_FORWARD_FOLDS = [
    ("2000-01-01", "2013-12-31", "2014-01-01", "2015-12-31", "2016-01-01",
"2017-12-31"),
    ("2000-01-01", "2015-12-31", "2016-01-01", "2017-12-31", "2018-01-01",
"2019-12-31"),
    ("2000-01-01", "2017-12-31", "2018-01-01", "2019-12-31", "2020-01-01",
"2021-12-31"),
    ("2000-01-01", "2019-12-31", "2020-01-01", "2021-12-31", "2022-01-01",
"2023-12-31"),
    ("2000-01-01", "2021-12-31", "2022-01-01", "2023-12-31", "2024-01-01",
"2026-07-29"),
]

TECHNICAL_FEATURES = [
    "RSI_14", "RSI_signal", "RSI_above_center",
    "MACD_line_pct", "MACD_signal_pct", "MACD_hist_pct", "MACD_cross",
    "EMA_12_26_ratio", "EMA_50_200_ratio",
    "price_to_EMA50_ratio", "price_to_EMA200_ratio",
```

```
    "StochRSI_K", "StochRSI_D", "StochRSI_cross",
]

MTF_FEATURES = ["RSI_weekly", "MACD_hist_weekly_pct"]

MACRO_FEATURES = [
    "vix_log", "bond_yield_diff", "gold_logret", "wti_logret",
    "fed_rate_diff", "m2_yoy"
]

FEATURES = TECHNICAL_FEATURES + MTF_FEATURES + MACRO_FEATURES

SCALE_COLS = [
    "MACD_line_pct", "MACD_signal_pct", "MACD_hist_pct",
    "EMA_12_26_ratio", "EMA_50_200_ratio",
    "price_to_EMA50_ratio", "price_to_EMA200_ratio",
    "MACD_hist_weekly_pct", "vix_log", "bond_yield_diff",
    "gold_logret", "wti_logret", "fed_rate_diff", "m2_yoy",
]



#
================================================================================
====
# 1. REPRODUCIBILITY & SYSTEM
#
================================================================================
====

def set_global_seed(seed: int):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)


def resolve_num_cpu(max_cap=8, reserve=1):
    try:
        logical = len(os.sched_getaffinity(0))
```

```
    except AttributeError:
        logical = os.cpu_count() or 1
    return max(1, min(max_cap, logical - reserve))


NUM_CPU = resolve_num_cpu(CFG.max_cpu, CFG.reserve_cpu)
N_STEPS = max(256, CFG.target_rollout // NUM_CPU)

if (N_STEPS * NUM_CPU) % CFG.batch_size != 0:
    raise ValueError(f"rollout={N_STEPS * NUM_CPU} ต้องหารด้วย
batch_size={CFG.batch_size} ลงตัว")

print(f"Device={CFG.device} | Workers={NUM_CPU} | n_steps/env={N_STEPS} |
rollout/update={N_STEPS * NUM_CPU}")


#
==========================================================================
====
# 2. FEATURE ENGINEERING
#
==========================================================================
====

def compute_complete_indicators(df, price_col="close"):
    df = df.copy()
    close = df[price_col].astype(float)

    rsi = ta.momentum.RSIIndicator(close=close, window=14).rsi()
    df["RSI_14"] = (rsi / 100.0) - 0.5
    df["RSI_signal"] = (rsi.rolling(9).mean() / 100.0) - 0.5
    df["RSI_above_center"] = (rsi > 50).astype(float)

    ema12 = ta.trend.EMAIndicator(close=close, window=12).ema_indicator()
    ema26 = ta.trend.EMAIndicator(close=close, window=26).ema_indicator()

    macd_obj = ta.trend.MACD(close=close, window_slow=26, window_fast=12,
window_sign=9)
    macd_line = macd_obj.macd()
    macd_signal = macd_obj.macd_signal()
```

```
    macd_hist = macd_obj.macd_diff()

    safe_ema26 = ema26.replace(0, np.nan)
    df["MACD_line_pct"] = macd_line / safe_ema26
    df["MACD_signal_pct"] = macd_signal / safe_ema26
    df["MACD_hist_pct"] = macd_hist / safe_ema26

    prev_diff = (macd_line - macd_signal).shift(1)
    curr_diff = macd_line - macd_signal
    df["MACD_cross"] = np.select(
        [(prev_diff <= 0) & (curr_diff > 0), (prev_diff >= 0) & (curr_diff
< 0)],
        [1.0, -1.0], default=0.0
    )

    ema50 = ta.trend.EMAIndicator(close=close, window=50).ema_indicator()
    ema200 = ta.trend.EMAIndicator(close=close,
window=200).ema_indicator()

    safe_ema50 = ema50.replace(0, np.nan)
    safe_ema200 = ema200.replace(0, np.nan)

    df["EMA_12_26_ratio"] = (ema12 - ema26) / safe_ema26
    df["EMA_50_200_ratio"] = (ema50 - ema200) / safe_ema200
    df["price_to_EMA50_ratio"] = (close - ema50) / safe_ema50
    df["price_to_EMA200_ratio"] = (close - ema200) / safe_ema200

    stochrsi = ta.momentum.StochRSIIndicator(close=close, window=14,
smooth1=3, smooth2=3)
    stoch_k = stochrsi.stochrsi_k()
    stoch_d = stochrsi.stochrsi_d()

    df["StochRSI_K"] = stoch_k - 0.5
    df["StochRSI_D"] = stoch_d - 0.5

    prev_stoch = (stoch_k - stoch_d).shift(1)
    curr_stoch = stoch_k - stoch_d

    bullish = (prev_stoch <= 0) & (curr_stoch > 0) & (stoch_k < 0.2)
    bearish = (prev_stoch >= 0) & (curr_stoch < 0) & (stoch_k > 0.8)
```

```
    df["StochRSI_cross"] = np.select([bullish, bearish], [1.0, -1.0],
default=0.0)
    return df


def compute_multi_timeframe_features(df, price_col="close",
date_col="date"):
    df = df.copy()
    df[date_col] = pd.to_datetime(df[date_col])
    df = df.sort_values(date_col).set_index(date_col)

    weekly = df[price_col].resample("W-FRI").last().dropna()

    weekly_rsi = ta.momentum.RSIIndicator(close=weekly, window=14).rsi()
    weekly_macd = ta.trend.MACD(close=weekly, window_slow=26,
window_fast=12, window_sign=9)
    weekly_macd_hist = weekly_macd.macd_diff()
    weekly_ema26 = ta.trend.EMAIndicator(close=weekly,
window=26).ema_indicator()
    weekly_macd_pct = weekly_macd_hist / weekly_ema26.replace(0, np.nan)

    weekly_features = pd.DataFrame({
        "RSI_weekly": (weekly_rsi / 100.0) - 0.5,
        "MACD_hist_weekly_pct": weekly_macd_pct,
    }, index=weekly.index).dropna(subset=["RSI_weekly",
"MACD_hist_weekly_pct"])

    weekly_features.index.name = date_col
    weekly_features = weekly_features.reset_index()
    weekly_features[date_col] = pd.to_datetime(weekly_features[date_col])

    daily = df.reset_index()
    daily[date_col] = pd.to_datetime(daily[date_col])

    daily = pd.merge_asof(
        daily.sort_values(date_col),
        weekly_features.sort_values(date_col),
        on=date_col,
        direction="backward",
```

```
    )
    return daily


#
================================================================================
====
# 3. DATA ACQUISITION
#
================================================================================
====

def _normalize_yahoo_columns(df):
    if isinstance(df.columns, pd.MultiIndex):
        df.columns = [c[0] for c in df.columns]
    else:
        df.columns = list(df.columns)
    return df


def fetch_raw_data(tickers, start_date, end_date):
    print(f"📦 กำลังดาวน์โหลดข้อมูล ETF: {start_date} -> {end_date}")
    processed = []

    for ticker in tickers:
        df = yf.download(ticker, start=start_date, end=end_date,
progress=False, auto_adjust=True, actions=False)
        if df.empty:
            raise ValueError(f"ไม่ได้รับข้อมูลจาก Yahoo Finance สำหรับ {ticker}")

        df = _normalize_yahoo_columns(df).reset_index()
        df.rename(
            columns={"Date": "date", "Open": "open", "High": "high",
"Low": "low", "Close": "close", "Volume": "volume"},
            inplace=True,
        )

        required = ["date", "open", "high", "low", "close", "volume"]
        missing = [c for c in required if c not in df.columns]
        if missing:
```

```
            raise ValueError(f"{ticker}: ขาดคอลัมน์ {missing}")

        df["tic"] = ticker
        processed.append(df[["date", "tic", "open", "high", "low",
"close", "volume"]])

    final_df = pd.concat(processed, ignore_index=True)
    final_df["date"] =
pd.to_datetime(final_df["date"]).dt.tz_localize(None)

    # ดาวน์โหลดข้อมูล Macro จาก Yahoo Finance
    macro_tickers = {"^VIX": "vix_raw", "^TNX": "bond_yield_raw", "GC=F":
"gold_raw", "CL=F": "wti_raw"}
    raw_macro = yf.download(list(macro_tickers.keys()), start=start_date,
end=end_date, progress=False, auto_adjust=False, actions=False)
    if raw_macro.empty:
        raise ValueError("ไม่พบข้อมูล Macro จาก Yahoo")

    close = raw_macro["Close"].copy() if isinstance(raw_macro.columns,
pd.MultiIndex) else raw_macro[["Close"]].copy()
    if isinstance(close.columns, pd.MultiIndex):
        close.columns = [c[0] for c in close.columns]

    close.rename(columns=macro_tickers, inplace=True)
    macro_df = close.reset_index().rename(columns={"Date": "date"})
    macro_df["date"] =
pd.to_datetime(macro_df["date"]).dt.tz_localize(None)

    macro_df["vix_log_raw"] = np.log(macro_df["vix_raw"].clip(lower=1e-3))
    macro_df["vix_log"] = macro_df["vix_log_raw"]
    macro_df["bond_yield_diff"] = macro_df["bond_yield_raw"].diff()
    macro_df["gold_logret"] =
np.log(macro_df["gold_raw"].clip(lower=1e-3)).diff().clip(-0.5, 0.5)
    macro_df["wti_logret"] =
np.log(macro_df["wti_raw"].clip(lower=1e-3)).diff().clip(-0.5, 0.5)

    final_df = pd.merge_asof(
        final_df.sort_values("date"),
        macro_df.sort_values("date"),
        on="date",
```

```
        direction="backward",
    )

    # ดาวน์โหลดข้อมูล Macro จาก FRED
    try:
        fed = web.DataReader("FEDFUNDS", "fred", start_date,
end_date).reset_index()
        m2 = web.DataReader("M2SL", "fred", start_date,
end_date).reset_index()

        fed.columns = ["date", "fed_raw"]
        m2.columns = ["date", "m2_raw"]

        fed["date"] = pd.to_datetime(fed["date"])
        m2["date"] = pd.to_datetime(m2["date"])

        fed["fed_rate_diff"] = fed["fed_raw"].diff()
        m2["m2_yoy"] = m2["m2_raw"].pct_change(12)

        fed["date"] += pd.Timedelta(days=CFG.fed_funds_lag_days)
        m2["date"] += pd.Timedelta(days=CFG.m2_lag_days)

        final_df = pd.merge_asof(
            final_df.sort_values("date"),
            fed[["date", "fed_rate_diff"]].sort_values("date"),
            on="date",
            direction="backward",
        )
        final_df = pd.merge_asof(
            final_df.sort_values("date"),
            m2[["date", "m2_yoy"]].sort_values("date"),
            on="date",
            direction="backward",
        )
        print("✅ ข้อมูล FRED โหลดสำเร็จพร้อม Conservative Lag")
    except Exception as exc:
        raise RuntimeError("การดาวน์โหลดข้อมูล FRED ล้มเหลว กรุณาตรวจสอบการเชื่อมต่ออินเทอร์เน็ต") from exc
```

```
    final_df = final_df.sort_values(["date",
"tic"]).reset_index(drop=True)
    if final_df[["open", "close"]].isna().any().any():
        raise ValueError("พบค่า NaN ในราคา Open/Close")

    return final_df


#
================================================================================
====
# 4. FEATURE PIPELINE
#
================================================================================
====

def process_features_for_slice(raw_df, target_start, target_end,
label=""):
    target_start = pd.Timestamp(target_start)
    target_end = pd.Timestamp(target_end)

    sub_df = raw_df[raw_df["date"] <= target_end].copy()
    processed = []

    for tic, group in sub_df.groupby("tic", sort=False):
        g = group.sort_values("date").copy()
        g = compute_complete_indicators(g, "close")
        g = compute_multi_timeframe_features(g, "close", "date")
        processed.append(g)

    if not processed:
        raise ValueError(f"ไม่มีข้อมูลสำหรับ {target_start} -> {target_end}")

    full_slice = pd.concat(processed, ignore_index=True)
    full_slice[FEATURES] = full_slice.groupby("tic",
sort=False)[FEATURES].ffill()
    full_slice = full_slice.dropna(subset=FEATURES).copy()

    out = full_slice[(full_slice["date"] >= target_start) &
(full_slice["date"] <= target_end)].copy()
```

```
    if out.empty:
        raise ValueError(f"{label}: ไม่มีแถวข้อมูลที่สมบูรณ์ในช่วง {target_start}
-> {target_end}")

    counts = out.groupby("date")["tic"].nunique()
    valid_dates = counts[counts == len(CFG.tickers)].index
    out = out[out["date"].isin(valid_dates)].copy()

    if out.empty:
        raise ValueError(f"{label}: ไม่มีวันที่ที่มี Ticker ครบทั้งหมด
{CFG.tickers}")

    dates = sorted(out["date"].unique())
    day_map = {d: i for i, d in enumerate(dates)}
    out["day"] = out["date"].map(day_map)
    out = out.sort_values(["day", "tic"]).reset_index(drop=True)

    if out.duplicated(["day", "tic"]).any():
        raise ValueError(f"{label}: พบแถว day/ticker ซ้ำซ้อน")

    if label:
        print(f"  {label:<5}: {dates[0].date()} -> {dates[-1].date()}
({len(dates)} วันทำการ)")

    return out


def compute_scaling_stats(train_df):
    stats = {}
    for col in SCALE_COLS:
        if col not in train_df.columns:
            continue
        x = train_df[col].replace([np.inf, -np.inf], np.nan).dropna()
        if len(x) < 2:
            stats[col] = (0.0, 1.0)
            continue
        stats[col] = (float(x.mean()), float(x.std()) if float(x.std()) >
1e-6 else 1.0)
    return stats
```

```
def apply_scaling(df, stats, clip_sigma=5.0):
    df = df.copy()
    for col, (mean, std) in stats.items():
        if col in df.columns:
            df[col] = ((df[col] - mean) / std).clip(-clip_sigma,
clip_sigma)
    return df


#
================================================================================
====
# 5. ENVIRONMENT (Zero Leakage, Open-to-Open Holding)
#
================================================================================
====

class PortfolioWeightTradingEnv(gym.Env):
    """
    Continuous Multi-day Holding Horizon (Open-to-Open):
        observation(t) = ข้อมูลสถานะหลังตลาดปิด Close(t)
        action(t)      = สัดส่วนน้ำหนักพอร์ตเป้าหมาย
        execution      = Rebalance ที่ Open(t+1)
        valuation      = วัดมูลค่าต่อเนื่องถึง Open(t+2) (รักษา overnight return)

    Zero-Leakage Guarantee:
        Observation ณ วันที่ t+1 (Close(t+1)) จะใช้ weights ที่ drift ตามราคา
Close(t+1)
        เท่านั้น จะไม่นำ weights ณ Open(t+2) มาต่อใน State เด็ดขาด
    """

    metadata = {"render_modes": []}

    def __init__(
        self,
        df,
        tickers,
        initial_amount=CFG.initial_amount,
        transaction_cost_pct=CFG.transaction_cost_pct,
```

```
    ):
        super().__init__()
        self.df = df.copy()
        self.tickers = tuple(tickers)
        self.k = len(self.tickers)

        self.initial_amount = float(initial_amount)
        self.cost_pct = float(transaction_cost_pct)

        self.dates = sorted(self.df["date"].unique())
        self.total_days = len(self.dates)

        if self.total_days < 3:
            raise ValueError("Environment ต้องการข้อมูลอย่างน้อย 3 วันทำการสำหรับการเทรดแบบ Open-to-Open")

        obs_dim = self.k * len(FEATURES) + (self.k + 1)
        self.observation_space = spaces.Box(
            low=-np.inf, high=np.inf, shape=(obs_dim,), dtype=np.float32
        )
        self.action_space = spaces.Box(
            low=-5.0, high=5.0, shape=(self.k + 1,), dtype=np.float32
        )

        self._build_aligned_matrices()

    def _build_aligned_matrices(self):
        open_pivot = (
            self.df.pivot_table(index="day", columns="tic", values="open", aggfunc="last")
            .reindex(columns=self.tickers)
        )
        close_pivot = (
            self.df.pivot_table(index="day", columns="tic", values="close", aggfunc="last")
            .reindex(columns=self.tickers)
        )

        if open_pivot.isna().any().any() or close_pivot.isna().any().any():
```

```
            raise ValueError("พบค่า NaN ใน Open/Close Matrix")

        self.open_matrix = open_pivot.to_numpy(dtype=np.float64)
        self.close_matrix = close_pivot.to_numpy(dtype=np.float64)

        if np.any(self.open_matrix <= 0) or np.any(self.close_matrix <=
0):
            raise ValueError("Open/Close ทั้งหมดต้องเป็นค่าบวก")

        feat_list = []
        for tic in self.tickers:
            t_df = self.df[self.df["tic"] == tic].sort_values("day")
            if len(t_df) != self.total_days:
                raise ValueError(f"{tic}: คาดหวัง {self.total_days} แถว แต่
พบ {len(t_df)}")
            feat_list.append(t_df[FEATURES].to_numpy(dtype=np.float32))

        self.feat_matrix = np.stack(feat_list, axis=1)

        t0 = self.df[self.df["tic"] == self.tickers[0]].sort_values("day")
        if "vix_log_raw" not in t0.columns:
            raise ValueError("Environment ต้องการ vix_log_raw สำหรับ reward
regime audit")
        self.vix_raw_log = t0["vix_log_raw"].to_numpy(dtype=np.float64)

        if len(self.vix_raw_log) != self.total_days:
            raise ValueError("VIX matrix length ไม่ตรงกับจำนวนวัน")

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)

        self.current_day = 0
        self.portfolio_value = float(self.initial_amount)
        self.peak_value = float(self.initial_amount)
        self.prev_dd = 0.0

        # Weights ตอนถือครองจริง (สำหรับ rebalance turnover ที่ Open)
        self.weights = np.zeros(self.k + 1, dtype=np.float64)
        self.weights[0] = 1.0
```

```
        # Weights สำหรับต่อ Observation Vector (ณ จุดเวลา Close)
        self.obs_weights = self.weights.copy()

        self.asset_memory = [self.portfolio_value]
        # เริ่มต้นบันทึกวันที่ก้าวแรกเริ่มมีผล (Open(1)) เพื่อให้ Memory เรียงวันต่อเนื่องไม่มี Gap
        self.date_memory = [self.dates[1]]

        obs = self._get_obs(self.current_day)
        info = {
            "date": self.dates[self.current_day],
            "portfolio_value": self.portfolio_value,
            "weights": self.weights.astype(np.float32),
        }
        return obs, info

    def _get_obs(self, day_idx):
        if not 0 <= day_idx < self.total_days:
            raise IndexError(f"day_idx={day_idx} อยู่นอกช่วง
0..{self.total_days - 1}")
        feats = self.feat_matrix[day_idx].reshape(-1)
        return np.concatenate([feats,
self.obs_weights]).astype(np.float32)

    @staticmethod
    def _softmax(actions):
        actions = np.asarray(actions, dtype=np.float64)
        actions = np.clip(actions, -5.0, 5.0)
        shifted = actions - np.max(actions)
        exp_a = np.exp(shifted)
        denom = np.sum(exp_a)
        if not np.isfinite(denom) or denom <= 0:
            raise FloatingPointError("Softmax denominator ผิดปกติ")
        return exp_a / denom

    def _transition(self, target_weights):
        """
        Pure accounting transition:
        Rebalance ที่ Open(t+1) -> วัดมูลค่าบัญชีถึง Open(t+2)
        """
        target_weights = np.asarray(target_weights, dtype=np.float64)
```

```
        if target_weights.shape != (self.k + 1,):
            raise ValueError("target_weights shape ไม่ถูกต้อง")
        if np.any(target_weights < -1e-12):
            raise ValueError("target_weights ต้องไม่ติดลบ")
        if not np.isclose(target_weights.sum(), 1.0, atol=1e-10):
            raise ValueError("target_weights ต้องรวมได้ 1")

        exec_open = self.open_matrix[self.current_day + 1]
        next_open = self.open_matrix[self.current_day + 2]

        if np.any(exec_open <= 0) or np.any(next_open <= 0):
            raise ValueError("Open prices ต้องเป็นค่าบวก")

        # Turnover คิดเฉพาะสินทรัพย์เสี่ยง (Cash เป็น residual)
        turnover = float(np.sum(np.abs(target_weights[1:] -
self.weights[1:])))
        rebalance_cost = float(self.portfolio_value * turnover *
self.cost_pct)
        rebalance_cost = min(max(rebalance_cost, 0.0),
self.portfolio_value)

        # Open(t+1) ถึง Open(t+2)
        price_relatives = np.concatenate([[1.0], next_open / exec_open])

        net_capital = self.portfolio_value - rebalance_cost
        new_asset_values = net_capital * target_weights * price_relatives
        new_portfolio_value = float(np.sum(new_asset_values))

        if not np.isfinite(new_portfolio_value) or new_portfolio_value <=
0:
            raise FloatingPointError(f"มูลค่าพอร์ตผิดปกติ:
{new_portfolio_value}")

        valuation_weights = new_asset_values / new_portfolio_value
        step_return = (new_portfolio_value - self.portfolio_value) /
self.portfolio_value

        return (
            new_portfolio_value,
```

```
            valuation_weights,
            float(step_return),
            turnover,
            rebalance_cost,
        )

    def step(self, actions):
        if self.current_day >= self.total_days - 2:
            raise RuntimeError("step() ถูกเรียกหลังจาก Episode จบแล้ว")

        target_weights = self._softmax(actions)

        # No-trade band: ถ้าน้ำหนักเปลี่ยนน้อยกว่า threshold จะคงพอร์ตเดิมเพื่อประหยัด
Cost Churn
        prospective_turnover = float(
            np.sum(np.abs(target_weights[1:] - self.weights[1:]))
        )
        if prospective_turnover < CFG.min_turnover_threshold:
            target_weights = self.weights.copy()

        decision_vix_log = self.vix_raw_log[self.current_day]

        (
            new_portfolio_value,
            valuation_weights,
            step_return,
            turnover,
            rebalance_cost,
        ) = self._transition(target_weights)

        self.portfolio_value = new_portfolio_value
        self.weights = valuation_weights  # อัปเดต holding weights สำหรับรอบ
rebalance ถัดไป
        self.peak_value = max(self.peak_value, self.portfolio_value)

        # คำนวณ Reward Shaping
        reward = float(np.log(max(1e-8, 1.0 + step_return)) * 100.0)

        current_dd = max(0.0, 1.0 - (self.portfolio_value /
self.peak_value))
```

```
        drawdown_increase = max(0.0, current_dd - self.prev_dd)
        reward -= drawdown_increase * 100.0 * CFG.dd_penalty_mult
        self.prev_dd = current_dd

        invested_ratio = 1.0 - float(valuation_weights[0])
        if invested_ratio < CFG.cash_drag_threshold:
            drag_multiplier = (
                CFG.high_vix_drag_multiplier
                if decision_vix_log > CFG.high_vix_threshold
                else 1.0
            )
            reward -= (
                CFG.cash_drag_strength
                * (CFG.cash_drag_threshold - invested_ratio)
                * drag_multiplier
            )

        #
--------------------------------------------------------------------------
        # ป้องกัน Data Leakage: คำนวณสัดส่วนพอร์ต ณ Close(t+1) สำหรับ
Observation วันพรุ่งนี้
        #
--------------------------------------------------------------------------
        exec_open = self.open_matrix[self.current_day + 1]
        close_today = self.close_matrix[self.current_day + 1]
        net_cap = self.portfolio_value / (1.0 + step_return)  # ทุนสุทธิหลัง
หัก cost ตอน Open(t+1)
        close_rel = np.concatenate([[1.0], close_today / exec_open])
        close_asset_vals = net_cap * target_weights * close_rel
        self.obs_weights = close_asset_vals / np.sum(close_asset_vals)

        self.current_day += 1
        truncated = self.current_day >= (self.total_days - 2)
        terminated = False

        value_date = self.dates[self.current_day + 1]
        self.asset_memory.append(self.portfolio_value)
        self.date_memory.append(value_date)

        info = {
```

```
            "date": value_date,
            "portfolio_value": self.portfolio_value,
            "step_return": step_return,
            "turnover": turnover,
            "transaction_cost": rebalance_cost,
            "drawdown": current_dd,
            "invested_ratio": invested_ratio,
            "weights": valuation_weights.astype(np.float32),
            "target_weights": target_weights.astype(np.float32),
            "execution_price_basis": "Open(t+1)",
            "valuation_price_basis": "Open(t+2)",
        }

        obs = self._get_obs(self.current_day)
        return obs, reward, terminated, truncated, info

    def save_asset_memory(self):
        return pd.DataFrame({"date": self.date_memory, "account_value":
self.asset_memory})


#
================================================================================
====
# 6. PICKLABLE ENVIRONMENT FACTORY
#
================================================================================
====

class EnvFactory:
    def __init__(self, df, tickers, rank, base_seed):
        self.df = df
        self.tickers = tickers
        self.rank = rank
        self.base_seed = base_seed

    def __call__(self):
        env = PortfolioWeightTradingEnv(df=self.df, tickers=self.tickers)
        env.reset(seed=self.base_seed + self.rank)
        return env
```

```
#
================================================================================
====
# 7. AGENT BUILDER
#
================================================================================
====

def build_agent(env, seed):
    policy_kwargs = dict(
        net_arch=dict(pi=[128, 128], vf=[128, 128]),
        activation_fn=nn.Tanh,
        ortho_init=True,
        log_std_init=CFG.log_std_init,
    )
    return PPO(
        "MlpPolicy",
        env,
        learning_rate=CFG.learning_rate,
        n_steps=N_STEPS,
        batch_size=CFG.batch_size,
        n_epochs=CFG.n_epochs,
        gamma=CFG.gamma,
        gae_lambda=CFG.gae_lambda,
        vf_coef=CFG.vf_coef,
        ent_coef=CFG.ent_coef,
        clip_range=CFG.clip_range,
        max_grad_norm=CFG.max_grad_norm,
        target_kl=CFG.target_kl,
        policy_kwargs=policy_kwargs,
        device=CFG.device,
        seed=seed,
        verbose=0,
    )
```

```
#
================================================================================
====
# 8. BACKTEST ENGINE
#
================================================================================
====

def run_backtest(model, test_df, tickers):
    env = PortfolioWeightTradingEnv(df=test_df, tickers=tickers)
    obs, _ = env.reset(seed=CFG.seed)
    rows = []

    while True:
        action, _ = model.predict(obs, deterministic=True)
        obs, reward, terminated, truncated, info = env.step(action)

        rows.append({
            "date": info["date"],
            "account_value": info["portfolio_value"],
            "daily_return": info["step_return"],
            "turnover": info["turnover"],
            "transaction_cost": info["transaction_cost"],
            "drawdown": info["drawdown"],
            "invested_ratio": info["invested_ratio"],
            "cash_weight": float(info["weights"][0]),
            "DIA_weight": float(info["weights"][1]),
            "QQQ_weight": float(info["weights"][2]),
            "SPY_weight": float(info["weights"][3]),
        })

        if terminated or truncated:
            break

    acct = pd.DataFrame(rows)
    acct["date"] = pd.to_datetime(acct["date"])
    return acct.sort_values("date").reset_index(drop=True)
```

```
#
================================================================================
====
# 9. FAIR BENCHMARK (Friction-Matched & Timeline-Aligned)
#
================================================================================
====

def _benchmark_series_open_to_open(test_df, tickers):
    """
    สร้างซีรีส์ผลตอบแทน Benchmark โดยวางจังหวะเวลาตรงกับ Agent แบบ 100%:
    Agent ตัดสินใจที่ Close(0) -> Rebalance ที่ Open(1) -> วัดมูลค่าที่ Open(2)
    ดังนั้น Benchmark จะเริ่มคำนวณผลตอบแทนก้าวแรกจาก Open(1) ไปยัง Open(2) เช่นเดียวกัน
    """
    open_pivot = (
        test_df.pivot_table(index="date", columns="tic", values="open",
aggfunc="last")
        .reindex(columns=tickers)
        .dropna()
    )

    if len(open_pivot) < 3:
        return None, None

    # SPY Buy & Hold: เริ่มจาก Cash 100%, ซื้อที่ Open(1) พร้อมจ่ายค่าธรรมเนียมก้าวแรก
    spy_open = open_pivot["SPY"].to_numpy(dtype=np.float64)
    spy_rows = []
    for i in range(1, len(spy_open) - 1):
        gross = spy_open[i + 1] / spy_open[i]
        if i == 1:
            gross *= (1.0 - CFG.transaction_cost_pct)
        spy_rows.append(gross - 1.0)
    spy_ret = pd.Series(spy_rows, index=open_pivot.index[2:],
name="SPY_bh")

    # Equal-Weight: เริ่มจาก Cash 100%, ปรับสัดส่วนเท่ากันทุกวันพร้อมหักค่าธรรมเนียม
    n = len(tickers)
    target = np.full(n, 1.0 / n, dtype=np.float64)
    values = 1.0
    current_weights = np.zeros(n, dtype=np.float64)
```

```
    eq_rows = []

    for i in range(1, len(open_pivot) - 1):
        turnover = float(np.sum(np.abs(target - current_weights)))
        cost = values * turnover * CFG.transaction_cost_pct
        net_val = values - min(max(cost, 0.0), values)

        rel = (
            open_pivot.iloc[i + 1].to_numpy(dtype=np.float64)
            / open_pivot.iloc[i].to_numpy(dtype=np.float64)
        )
        new_asset_vals = (net_val * target) * rel
        new_total_val = float(np.sum(new_asset_vals))

        if not np.isfinite(new_total_val) or new_total_val <= 0:
            raise FloatingPointError("Benchmark account value ผิดปกติ")

        period_return = (new_total_val - values) / values
        values = new_total_val
        current_weights = new_asset_vals / new_total_val

        eq_rows.append({
            "date": open_pivot.index[i + 1],
            "daily_return": period_return,
            "turnover": turnover,
            "transaction_cost": cost,
            "account_value": values,
        })

    eq = pd.DataFrame(eq_rows)
    return spy_ret, eq


def benchmark_open_to_open(test_df, tickers):
    spy_ret, eq = _benchmark_series_open_to_open(test_df, tickers)
    if spy_ret is None:
        return None, None

    return (
        compute_metrics(spy_ret, "B&H SPY Open-to-Open + Entry Cost"),
```

```
        compute_metrics(eq["daily_return"], "Equal-Weight Open-to-Open +
Cost (Cash Start)"),
    )


#
================================================================================
====
# 10. METRICS EVALUATION
#
================================================================================
====

def compute_metrics(returns, label=""):
    if returns is None:
        return None

    returns = pd.Series(returns).replace([np.inf, -np.inf],
np.nan).dropna()
    if len(returns) < 2:
        return None

    return {
        "fold": label,
        "days": int(len(returns)),
        "total_return": float(qs.stats.comp(returns) * 100),
        "cagr": float(qs.stats.cagr(returns) * 100),
        "max_dd": float(qs.stats.max_drawdown(returns) * 100),
        "sharpe": float(qs.stats.sharpe(returns)),
        "sortino": float(qs.stats.sortino(returns)),
        "win_rate": float(qs.stats.win_rate(returns) * 100),
        "volatility": float(qs.stats.volatility(returns) * 100),
        "calmar": float(qs.stats.calmar(returns)),
    }


def summarize_account_metrics(acct, label):
    metrics = compute_metrics(acct["daily_return"], label)
    if metrics is None:
        return None
```

```
    metrics["avg_turnover"] = float(acct["turnover"].mean())
    metrics["total_transaction_cost"] =
float(acct["transaction_cost"].sum())
    metrics["avg_cash"] = float(acct["cash_weight"].mean())
    metrics["avg_invested"] = float(acct["invested_ratio"].mean())
    metrics["max_cash"] = float(acct["cash_weight"].max())
    return metrics


#
================================================================================
====
# 11. WALK-FORWARD PIPELINE
#
================================================================================
====

def run_fold(fold_id, dates, raw_df, out_dir):
    tr_s, tr_e, v_s, v_e, te_s, te_e = dates

    print("\n" + "=" * 78)
    print(f"FOLD {fold_id} | Train[{tr_s}:{tr_e}] | Val[{v_s}:{v_e}] |
Test[{te_s}:{te_e}]")
    print("=" * 78)

    train_df = process_features_for_slice(raw_df, tr_s, tr_e, "TRAIN")
    val_df = process_features_for_slice(raw_df, v_s, v_e, "VAL")
    test_df = process_features_for_slice(raw_df, te_s, te_e, "TEST")

    # Train-only Scaling
    stats = compute_scaling_stats(train_df)
    train_df = apply_scaling(train_df, stats)
    val_df = apply_scaling(val_df, stats)
    test_df = apply_scaling(test_df, stats)

    fold_dir = Path(out_dir) / f"fold_{fold_id}"
    fold_dir.mkdir(parents=True, exist_ok=True)
```

```
    pd.DataFrame([{"feature": col, "mean": m, "std": s} for col, (m, s) in
stats.items()]).to_csv(
        fold_dir / "train_scaling_stats.csv", index=False
    )

    env_fns = [EnvFactory(train_df, CFG.tickers, i, CFG.seed + fold_id *
1000) for i in range(NUM_CPU)]
    env_train_raw = SubprocVecEnv(env_fns, start_method="spawn")
    env_val_raw = DummyVecEnv([lambda: PortfolioWeightTradingEnv(val_df,
CFG.tickers)])

    env_train = VecNormalize(env_train_raw, norm_obs=False,
norm_reward=True, clip_reward=10.0, gamma=CFG.gamma)
    env_val = VecNormalize(env_val_raw, norm_obs=False, norm_reward=False,
training=False)

    eval_cb = EvalCallback(
        env_val,
        best_model_save_path=str(fold_dir),
        log_path=str(fold_dir / "eval_logs"),
        eval_freq=max(1, CFG.eval_every // NUM_CPU),
        n_eval_episodes=CFG.n_eval_episodes,
        deterministic=True,
        verbose=0,
    )

    model = build_agent(env_train, seed=CFG.seed + fold_id)
    last_model_path = fold_dir / "last_model.zip"

    try:
        model.learn(total_timesteps=CFG.total_timesteps, callback=eval_cb)
    finally:
        model.save(str(last_model_path))
        env_train.close()
        env_val.close()

    best_model_path = fold_dir / "best_model.zip"
    target_model_path = best_model_path if best_model_path.exists() else
last_model_path
    model = PPO.load(str(target_model_path), device=CFG.device)
```

```
    # ทดสอบ Out-of-Sample บน Test Set
    acct = run_backtest(model, test_df, CFG.tickers)
    acct.to_csv(fold_dir / "backtest_account.csv", index=False)

    ai_metrics = summarize_account_metrics(acct, f"Fold{fold_id}
{te_s[:4]}-{te_e[:4]}")
    spy_bh, eq_bh = benchmark_open_to_open(test_df, CFG.tickers)

    benchmark_rows = [b for b in [spy_bh, eq_bh] if b]
    pd.DataFrame(benchmark_rows).to_csv(fold_dir /
"benchmark_metrics.csv", index=False)

    if ai_metrics:
        print(f"AI  : CAGR {ai_metrics['cagr']:7.2f}% | MaxDD
{ai_metrics['max_dd']:7.2f}% | Sharpe {ai_metrics['sharpe']:6.2f} |
AvgCash {ai_metrics['avg_cash']:6.2%}")
    if eq_bh:
        print(f"EqW : CAGR {eq_bh['cagr']:7.2f}% | MaxDD
{eq_bh['max_dd']:7.2f}% | Sharpe {eq_bh['sharpe']:6.2f}")
        if ai_metrics:
            print(f"ส่วนต่าง (Edge vs EqW): {ai_metrics['cagr'] -
eq_bh['cagr']:+.2f}% CAGR")

    return ai_metrics, eq_bh



#
================================================================================
====
# 12. DETERMINISTIC RESEARCH AUDIT SUITE (V9.2 Full Version)
#
================================================================================
====

def _make_synthetic_env(n_days=8, vix_by_day=None):
    dates = pd.bdate_range("2025-01-02", periods=n_days)
    rows = []

    for day, date in enumerate(dates):
```

```
        opens = {"DIA": 100.0 + day, "QQQ": 200.0 + 2.0 * day, "SPY":
300.0 + 3.0 * day}
        vix_level = vix_by_day[day] if vix_by_day is not None else 15.0

        for j, tic in enumerate(CFG.tickers):
            row = {
                "date": date,
                "tic": tic,
                "open": opens[tic],
                "high": opens[tic] * 1.01,
                "low": opens[tic] * 0.99,
                "close": opens[tic] * 1.005,
                "volume": 1000.0,
                "vix_log_raw": np.log(vix_level),
            }
            for feature in FEATURES:
                row[feature] = float((day + 1) * 0.001 + j * 0.0001)
            rows.append(row)

    df = pd.DataFrame(rows)
    dates_map = {d: i for i, d in enumerate(sorted(df["date"].unique()))}
    df["day"] = df["date"].map(dates_map)
    return df.sort_values(["day", "tic"]).reset_index(drop=True)


def _weights_to_action(weights):
    weights = np.asarray(weights, dtype=np.float64)
    if np.any(weights <= 0) or not np.isclose(weights.sum(), 1.0):
        raise ValueError("Audit weights must be strictly positive and sum
to 1.")
    return np.log(weights)


def audit_environment_accounting():
    """Test 1: ตรวจ accounting พื้นฐานของ environment"""
    df = _make_synthetic_env(n_days=5)
    env = PortfolioWeightTradingEnv(df, CFG.tickers)
    obs, info = env.reset(seed=CFG.seed)

    assert np.isclose(env.portfolio_value, CFG.initial_amount)
```

```
    assert np.isclose(env.weights[0], 1.0)
    assert np.allclose(env.weights[1:], 0.0)
    assert obs.shape == env.observation_space.shape
    assert len(env.asset_memory) == 1
    assert len(env.date_memory) == 1

    action = np.zeros(len(CFG.tickers) + 1, dtype=np.float64)
    old_value = env.portfolio_value
    obs2, reward, terminated, truncated, info = env.step(action)

    assert np.isfinite(info["portfolio_value"]) and
info["portfolio_value"] > 0
    expected_cost = old_value * info["turnover"] *
CFG.transaction_cost_pct
    assert np.isclose(info["transaction_cost"], expected_cost, atol=1e-8)
    assert np.isclose(np.sum(info["weights"]), 1.0, atol=1e-6)
    assert np.all(info["weights"] >= -1e-8)
    assert len(env.asset_memory) == 2
    assert len(env.date_memory) == 2
    assert obs2.shape == env.observation_space.shape


def audit_transition_validation():
    """Test 2: _transition ต้อง reject input ที่ผิดรูป"""
    df = _make_synthetic_env()
    env = PortfolioWeightTradingEnv(df, CFG.tickers)
    env.reset(seed=CFG.seed)

    bad_inputs = [
        np.array([1.0, 0.0, 0.0]),
        np.array([0.5, 0.5, 0.5, 0.5]),
        np.array([1.5, -0.5, 0.0, 0.0]),
    ]
    for bad in bad_inputs:
        try:
            env._transition(bad)
        except ValueError:
            pass
        else:
            raise AssertionError(f"_transition ต้อง reject input: {bad}")
```

```python
def audit_environment_timing():
    """Test 3: ตรวจสอบความถูกต้องทางคณิตศาสตร์และ Execution Timeline"""
    df = _make_synthetic_env()
    env = PortfolioWeightTradingEnv(df, CFG.tickers)
    env.reset(seed=CFG.seed)

    # Part A: Pure function
    target_pure = np.array([0.0, 0.0, 0.0, 1.0])
    old_value = env.portfolio_value
    new_value, new_weights, step_return, turnover, cost =
env._transition(target_pure)

    expected_cost = old_value * CFG.transaction_cost_pct
    spy_rel = env.open_matrix[2, 2] / env.open_matrix[1, 2]
    expected_value = (old_value - expected_cost) * spy_rel

    assert np.isclose(turnover, 1.0)
    assert np.isclose(cost, expected_cost)
    assert np.isclose(new_value, expected_value)
    assert np.isclose(step_return, expected_value / old_value - 1.0)

    # Part B: Step execution & Observation
    target_step = np.array([0.1, 0.1, 0.1, 0.7])
    exp_value, exp_weights, exp_ret, exp_to, exp_cost =
env._transition(target_step)

    obs2, reward, terminated, truncated, info =
env.step(_weights_to_action(target_step))

    assert env.current_day == 1
    assert info["date"] == env.dates[2]
    assert env.date_memory[-1] == env.dates[2]
    assert np.isclose(info["portfolio_value"], exp_value)
    assert np.isclose(info["step_return"], exp_ret)
    assert np.isclose(info["turnover"], exp_to)
    assert np.isclose(info["transaction_cost"], exp_cost)
    assert np.allclose(info["weights"], exp_weights.astype(np.float32),
atol=1e-6)
```

```
    assert not terminated
    assert obs2.shape == env.observation_space.shape


def audit_no_trade_band():
    """Test 4: ตรวจสอบ No-trade band ยกเลิกการเทรดเมื่อ Turnover ต่ำกว่าเกณฑ์"""
    df = _make_synthetic_env()
    env = PortfolioWeightTradingEnv(df, CFG.tickers)
    env.reset(seed=CFG.seed)

    action = np.zeros(4, dtype=np.float64)
    d = CFG.min_turnover_threshold / 10.0

    held = np.array([0.25 - 3 * d, 0.25 + d, 0.25 + d, 0.25 + d])
    env.weights = held.copy()
    old_value = env.portfolio_value

    obs, reward, terminated, truncated, info = env.step(action)

    assert np.isclose(info["turnover"], 0.0), "No-trade band ต้องยกเลิกการ
rebalance"
    assert np.isclose(info["transaction_cost"], 0.0)

    rel = np.concatenate([[1.0], env.open_matrix[2] / env.open_matrix[1]])
    expected_value = old_value * float(np.sum(held * rel))
    assert np.isclose(info["portfolio_value"], expected_value)


def audit_vix_drag_timing():
    """Test 5: Regime ของ VIX ต้องตัดสินที่วันตัดสินใจ (Close(t)) ไม่ใช่ t+1"""
    n_days = 6
    near_cash_action = np.array([5.0, -5.0, -5.0, -5.0])
    expected_base = -CFG.cash_drag_strength * CFG.cash_drag_threshold

    vix_a = [15.0, 40.0] + [15.0] * (n_days - 2)
    env_a = PortfolioWeightTradingEnv(_make_synthetic_env(n_days, vix_a),
CFG.tickers)
    env_a.reset(seed=CFG.seed)
    _, reward_a, _, _, info_a = env_a.step(near_cash_action)
```

```
    assert np.isclose(info_a["turnover"], 0.0)
    assert np.isclose(info_a["step_return"], 0.0)
    assert np.isclose(reward_a, expected_base * 1.0, atol=1e-9)

    vix_b = [40.0] + [15.0] * (n_days - 1)
    env_b = PortfolioWeightTradingEnv(_make_synthetic_env(n_days, vix_b),
CFG.tickers)
    env_b.reset(seed=CFG.seed)
    _, reward_b, _, _, info_b = env_b.step(near_cash_action)

    assert np.isclose(info_b["turnover"], 0.0)
    assert np.isclose(reward_b, expected_base *
CFG.high_vix_drag_multiplier, atol=1e-9)


def audit_terminal_state():
    """Test 6: ทดสอบการสิ้นสุด Episode ที่ total_days - 2"""
    df = _make_synthetic_env(n_days=4)
    env = PortfolioWeightTradingEnv(df, CFG.tickers)
    env.reset(seed=CFG.seed)

    action = np.zeros(4, dtype=np.float64)

    obs, reward, terminated, truncated, info = env.step(action)
    assert truncated is False
    assert env.current_day == 1
    assert info["date"] == env.dates[2]

    obs, reward, terminated, truncated, info = env.step(action)
    assert terminated is False
    assert truncated is True
    assert env.current_day == 2
    assert info["date"] == env.dates[3]
    assert env.date_memory[-1] == env.dates[3]
    assert obs.shape == env.observation_space.shape

    try:
        env.step(action)
    except RuntimeError:
        pass
```

```
    else:
        raise AssertionError("step() หลัง truncated ต้อง raise
RuntimeError")


def audit_benchmark_accounting():
    """Test 7: ตรวจสอบความถูกต้องของการตัดรอบบัญชี Benchmark"""
    df = _make_synthetic_env(n_days=6)
    spy_ret, eq = _benchmark_series_open_to_open(df, CFG.tickers)

    assert spy_ret is not None and eq is not None

    open_pivot = (
        df.pivot_table(index="date", columns="tic", values="open",
aggfunc="last")
        .reindex(columns=CFG.tickers)
        .dropna()
    )
    rel_step0 = (
        open_pivot.iloc[2].to_numpy(dtype=np.float64)
        / open_pivot.iloc[1].to_numpy(dtype=np.float64)
    )

    expected_eq_day1 = (
        (1.0 - 1.0 * CFG.transaction_cost_pct) * float(np.mean(rel_step0))
- 1.0
    )
    assert np.isclose(eq.iloc[0]["turnover"], 1.0), "EqW ก้าวแรกต้องมี
turnover = 1.0"
    assert eq.iloc[0]["transaction_cost"] > 0.0, "EqW ก้าวแรกต้องจ่าย initial
entry cost"
    assert np.isclose(eq.iloc[0]["daily_return"], expected_eq_day1,
atol=1e-12)

    spy_rel_step0 = float(open_pivot["SPY"].iloc[2] /
open_pivot["SPY"].iloc[1])
    expected_spy_day1 = spy_rel_step0 * (1.0 - CFG.transaction_cost_pct) -
1.0
    assert np.isclose(spy_ret.iloc[0], expected_spy_day1, atol=1e-12)
```

```
    spy_rel_step1 = float(open_pivot["SPY"].iloc[3] /
open_pivot["SPY"].iloc[2])
    assert np.isclose(spy_ret.iloc[1], spy_rel_step1 - 1.0, atol=1e-12),
"SPY B&H หลังวันแรกต้องไม่หัก cost ซ้ำ"
    assert eq.iloc[1]["turnover"] < 0.05


def audit_feature_cutoff_invariance():
    """Test 8: ป้องกันการรั่วไหลของ Feature ข้ามช่วงเวลา (Data Leakage Test)"""
    dates = pd.bdate_range("2020-01-01", periods=420)
    rng = np.random.default_rng(123)
    prices = 100.0 * np.exp(np.cumsum(rng.normal(0, 0.01, len(dates))))

    base = pd.DataFrame({"date": dates, "close": prices})
    full = compute_complete_indicators(base)
    full = compute_multi_timeframe_features(full)

    cutoff = dates[300]
    truncated_df = base[base["date"] <= cutoff].copy()
    trunc = compute_complete_indicators(truncated_df)
    trunc = compute_multi_timeframe_features(trunc)

    compare_cols = TECHNICAL_FEATURES + MTF_FEATURES
    a = full[full["date"] <= cutoff].set_index("date")[compare_cols]
    b = trunc.set_index("date")[compare_cols]

    common = a.index.intersection(b.index)
    for col in compare_cols:
        av = a.loc[common, col]
        bv = b.loc[common, col]
        mask = av.notna() & bv.notna()
        if mask.any():
            if not np.allclose(av[mask].to_numpy(), bv[mask].to_numpy(),
atol=1e-10, rtol=1e-8):
                raise AssertionError(f"Feature cutoff invariance ล้มเหลวที่
{col}")


def run_audit_suite():
    print("\n" + "=" * 78)
```

```
    print("🔍 V9.2 DETERMINISTIC RESEARCH AUDIT SUITE")
    print("=" * 78)

    tests = [
        ("environment accounting", audit_environment_accounting),
        ("transition validation", audit_transition_validation),
        ("environment timing", audit_environment_timing),
        ("no-trade band", audit_no_trade_band),
        ("vix drag timing", audit_vix_drag_timing),
        ("terminal state", audit_terminal_state),
        ("benchmark accounting", audit_benchmark_accounting),
        ("feature cutoff invariance", audit_feature_cutoff_invariance),
    ]

    for name, fn in tests:
        fn()
        print(f"  ✅ PASS: {name}")

    print("  ✅ ALL AUDITS PASSED SUCCESSFULLY")
    print("=" * 78)


#
================================================================================
====
# 13. MAIN EXECUTION
#
================================================================================
====

def main():
    set_global_seed(CFG.seed)
    out_dir = Path("./walkforward_v9_2_audited")
    out_dir.mkdir(parents=True, exist_ok=True)

    # รันการทดสอบความถูกต้องเชิงสถิติและคณิตศาสตร์การเงินทั้งหมดก่อนเทรนจริง
    run_audit_suite()

    print("\n🚀 กำลังเตรียมข้อมูล Master Dataset...")
```

```
    raw_df = fetch_raw_data(CFG.tickers, CFG.global_fetch_start,
CFG.global_end)

    all_metrics = []
    all_bh = []

    for fold_id, dates in enumerate(WALK_FORWARD_FOLDS, start=1):
        metrics, bh = run_fold(fold_id, dates, raw_df, out_dir)
        if metrics:
            all_metrics.append(metrics)
        if bh:
            all_bh.append(bh)

    if not all_metrics:
        raise RuntimeError("ไม่มีผลลัพธ์จาก Fold ใดเลย")

    ai_df = pd.DataFrame(all_metrics)
    ai_df.to_csv(out_dir / "walkforward_summary_ai.csv", index=False)

    bh_df = pd.DataFrame(all_bh)
    bh_df.to_csv(out_dir / "walkforward_summary_benchmark.csv",
index=False)

    print("\n" + "=" * 78)
    print("🏁 สรุปภาพรวม WALK-FORWARD VALIDATION (V9.2 AUDITED)")
    print("=" * 78)
    print(f"AI เฉลี่ย CAGR      : {ai_df['cagr'].mean():.2f}%")
    print(f"AI เฉลี่ย MaxDD     : {ai_df['max_dd'].mean():.2f}%")
    print(f"AI เฉลี่ย Sharpe    : {ai_df['sharpe'].mean():.2f}")
    print(f"AI Sharpe SD        : {ai_df['sharpe'].std(ddof=1):.2f}")
    print(f"AI เฉลี่ยถือเงินสด  : {ai_df['avg_cash'].mean():.2%}")
    print(f"AI เฉลี่ย Turnover  : {ai_df['avg_turnover'].mean():.4f}")


if __name__ == "__main__":
    main()
```

นี่คือผลลัพธ์ของโค้ด PPO_V2
Device=cpu | Workers=8 | n_steps/env=1024 | rollout/update=8192

==========
======
🔍 V9.2 DETERMINISTIC RESEARCH AUDIT SUITE
==========
======
  ✅ PASS: environment accounting
  ✅ PASS: transition validation
  ✅ PASS: environment timing
  ✅ PASS: no-trade band
  ✅ PASS: vix drag timing
  ✅ PASS: terminal state
  ✅ PASS: benchmark accounting
  ✅ PASS: feature cutoff invariance
  ✅ ALL AUDITS PASSED SUCCESSFULLY
==========
======

🚀 กำลังเตรียมข้อมูล Master Dataset...
📦 กำลังดาวน์โหลดข้อมูล ETF: 1996-01-01 -> 2026-07-29
✅ ข้อมูล FRED โหลดสำเร็จพร้อม Conservative Lag

==========
======
FOLD 1 | Train[2000-01-01:2013-12-31] | Val[2014-01-01:2015-12-31] |
Test[2016-01-01:2017-12-31]
==========
======
  TRAIN: 2000-08-31 -> 2013-12-31 (3353 วันทำการ)
  VAL : 2014-01-02 -> 2015-12-31 (504 วันทำการ)
  TEST : 2016-01-04 -> 2017-12-29 (503 วันทำการ)
AI : CAGR   6.87% | MaxDD   -5.72% | Sharpe   1.05 | AvgCash 36.81%
EqW : CAGR   20.59% | MaxDD -10.60% | Sharpe   1.86
ส่วนต่าง (Edge vs EqW): -13.72% CAGR

==========
======
FOLD 2 | Train[2000-01-01:2015-12-31] | Val[2016-01-01:2017-12-31] |
Test[2018-01-01:2019-12-31]
==========
======
  TRAIN: 2000-08-31 -> 2015-12-31 (3857 วันทำการ)

  VAL : 2016-01-04 -> 2017-12-29 (503 วันทำการ)
  TEST : 2018-01-02 -> 2019-12-31 (503 วันทำการ)
AI : CAGR   -0.93% | MaxDD -15.68% | Sharpe -0.07 | AvgCash 49.13%
EqW : CAGR   12.29% | MaxDD -19.64% | Sharpe   0.80
ส่วนต่าง (Edge vs EqW): -13.22% CAGR

==========
FOLD 3 | Train[2000-01-01:2017-12-31] | Val[2018-01-01:2019-12-31] |
Test[2020-01-01:2021-12-31]
==========
  TRAIN: 2000-08-31 -> 2017-12-29 (4360 วันทำการ)
  VAL : 2018-01-02 -> 2019-12-31 (503 วันทำการ)
  TEST : 2020-01-02 -> 2021-12-31 (505 วันทำการ)
AI : CAGR   17.59% | MaxDD -18.40% | Sharpe   1.10 | AvgCash 20.16%
EqW : CAGR   25.25% | MaxDD -31.54% | Sharpe   1.09
ส่วนต่าง (Edge vs EqW): -7.66% CAGR

==========
FOLD 4 | Train[2000-01-01:2019-12-31] | Val[2020-01-01:2021-12-31] |
Test[2022-01-01:2023-12-31]
==========
  TRAIN: 2000-08-31 -> 2019-12-31 (4863 วันทำการ)
  VAL : 2020-01-02 -> 2021-12-31 (505 วันทำการ)
  TEST : 2022-01-03 -> 2023-12-29 (501 วันทำการ)
AI : CAGR   -2.96% | MaxDD -23.65% | Sharpe -0.16 | AvgCash 38.20%
EqW : CAGR   2.18% | MaxDD -28.07% | Sharpe   0.21
ส่วนต่าง (Edge vs EqW): -5.14% CAGR

==========
FOLD 5 | Train[2000-01-01:2021-12-31] | Val[2022-01-01:2023-12-31] |
Test[2024-01-01:2026-07-29]
==========
  TRAIN: 2000-08-31 -> 2021-12-31 (5368 วันทำการ)
  VAL : 2022-01-03 -> 2023-12-29 (501 วันทำการ)
  TEST : 2024-01-02 -> 2026-07-28 (644 วันทำการ)
AI : CAGR   1.93% | MaxDD   -6.78% | Sharpe   0.45 | AvgCash 78.80%
EqW : CAGR   20.08% | MaxDD -20.11% | Sharpe   1.18
ส่วนต่าง (Edge vs EqW): -18.15% CAGR

==========
==========
🏁 สรุปภาพรวม WALK-FORWARD VALIDATION (V9.2 AUDITED)
==========
==========
AI เฉลี่ย CAGR : 4.50%
AI เฉลี่ย MaxDD : -14.05%
AI เฉลี่ย Sharpe : 0.47
AI Sharpe SD : 0.59
AI เฉลี่ยถือเงินสด : 44.62%
AI เฉลี่ย Turnover : 0.1152